

CPU Scheduling: Scheduling Times and Types

Scheduling Times

Scheduling times are metrics used to measure and compare the performance of CPU scheduling algorithms. They help evaluate how efficiently and fairly an operating system (OS) manages processes. Think of these as a report card for how well the OS handles tasks, like how long you wait at a restaurant or how quickly your food is served.

Arrival Time

- **Definition:** The time when a process enters the **Job Queue** in the **New** state, ready to be admitted to memory.
- **For Beginners:** Imagine arriving at a coffee shop and joining the line to place your order. The moment you step into the shop is your arrival time.
- **Example:** If you launch a web browser at 10:00:00 AM, its arrival time is 10:00:00.
- **Advanced:** Arrival time is recorded in the **Process Control Block (PCB)** when the process is created (e.g., via `fork()` in Linux). It's the starting point for calculating other metrics like turnaround time.
- **Notation:** Denoted as **AT**.

Burst Time

- **Definition:** The amount of time a process requires to complete its execution. Since we're assuming **no I/O operations** (as per the note), burst time refers only to **CPU Burst Time**.
- **For Beginners:** Think of burst time as how long it takes to complete a task uninterrupted, like baking a cake (pure work time, no waiting for ingredients).
- **Subtypes:**
 - **CPU Burst Time (CPU BT):**
 - The time a process needs to execute on the CPU to complete its task.

- **Example:** If a process (e.g., a program calculating prime numbers) needs 5 seconds of CPU time, its CPU BT = 5s.
- **Advanced:** In real systems, processes alternate between CPU bursts (computation) and I/O bursts (waiting for I/O). Since we assume no I/O, each process has one continuous CPU burst.
- **I/O Burst Time (IO BT):**
 - Not applicable here due to the assumption that processes have no I/O operations.
 - **For Context:** Normally, I/O burst time is the time spent waiting for I/O (e.g., reading a file). Without I/O, we focus solely on CPU BT.
- **Advanced:** Burst time is estimated or known in advance for some algorithms (e.g., Shortest Job First). In real OS, it's predicted based on past behavior (e.g., Linux uses historical data for scheduling).
- **Notation:** Denoted as **BT** (here, BT = CPU BT).

Completion Time

- **Definition:** The time when a process finishes execution and enters the **Terminated** state.
- **For Beginners:** This is when your coffee order is fully prepared and handed to you—your task is done.
- **Example:** If a process starts at 10:00:00 and finishes at 10:00:08, its completion time is 10:00:08.
- **Advanced:** Completion time depends on scheduling order, burst time, and waiting time in the Ready Queue. It's the endpoint for calculating turnaround time.
- **Notation:** Denoted as **CT**.

Turnaround Time

- **Definition:** The total time from when a process arrives (enters Job Queue) to when it completes (terminates). It includes:
 - Waiting time in the Ready Queue.
 - CPU execution time (Burst Time, since no I/O).

- **For Beginners:** Imagine the total time from entering the coffee shop to leaving with your coffee, including waiting in line and preparation time.
- **Example:** If a process arrives at 10:00:00, waits 3 seconds in the Ready Queue, and executes for 5 seconds, its turnaround time is $3 + 5 = 8$ seconds.
- **Formula for TAT:**
 - $TAT = CT - AT$
 - Turnaround Time = Completion Time - Arrival Time
 - **Explanation:** TAT measures the entire “journey” of a process through the system.
- **Advanced:** TAT is a key metric for batch systems, where minimizing total process time is critical. It varies by scheduling algorithm (e.g., Shortest Job First minimizes TAT for short processes).
- **Notation:** Denoted as **TAT**.

Waiting Time

- **Definition:** The total time a process spends in the **Ready Queue** waiting for the CPU (excludes execution time).
- **For Beginners:** This is the time you spend standing in line at the coffee shop, not including the time it takes to make your coffee.
- **Example:** If a process waits 3 seconds in the Ready Queue before getting 5 seconds of CPU time, its waiting time is 3 seconds.
- **Formula for WT:**
 - $WT = TAT - BT$
 - Waiting Time = Turnaround Time - Burst Time
 - **Explanation:** Since TAT includes both waiting and execution, subtract the execution time (BT) to get waiting time.
- **Advanced:** Waiting time is critical for interactive systems (e.g., apps responding to clicks). Preemptive algorithms like Round-Robin reduce waiting time for fairness but may increase context switch overhead.
- **Notation:** Denoted as **WT**.

Response Time

- **Definition:** The time from when a process arrives (enters Job Queue) to when it first gets CPU time (starts execution).
- **For Beginners:** This is how long you wait at the coffee shop before the barista starts working on your order (not when it's finished).
- **Example:** If a process arrives at 10:00:00 and first gets the CPU at 10:00:02, its response time is 2 seconds.
- **Advanced:** Response time is crucial for interactive systems (e.g., games or GUIs) where quick initial response feels snappy. In preemptive systems, response time is often lower than in non-preemptive ones, as processes can start sooner.
- **Notation:** Denoted as **RT**.

Scheduling Length

- **Definition:** The time span from the earliest arrival time of any process to the latest completion time of any process in a set of processes.
- **For Beginners:** Imagine a coffee shop opening at 9 AM (first customer arrives) and finishing the last order at 10 AM. The scheduling length is 1 hour.
- **Formula:**
 - **Scheduling Length = $\max(CT) - \min(AT)$**
 - Maximum Completion Time - Minimum Arrival Time
- **Example:** For three processes with $AT = \{0, 1, 2\}$ seconds and $CT = \{5, 7, 8\}$ seconds, scheduling length = $\max(8) - \min(0) = 8$ seconds.
- **Advanced:** Scheduling length measures the total duration of a scheduling run. It's useful for evaluating system efficiency in batch processing, where shorter length means faster overall completion.
- **Notation:** Not typically abbreviated, but sometimes called **makespan**.

Throughput

- **Definition:** The number of processes completed per unit of time.
- **For Beginners:** Think of how many coffees the shop serves in an hour. More coffees = higher throughput.
- **Formula:**
 - **Throughput = Number of Processes Executed / Time Unit**

- **Example:** If 10 processes complete in 20 seconds, throughput = $10 / 20 = 0.5$ processes per second.
- **Example:** If an OS completes 5 apps (e.g., opening browsers, saving files) in 10 seconds, throughput is $5 / 10 = 0.5$ processes/second.
- **Advanced:** High throughput indicates efficient CPU use, especially in batch systems. Algorithms like Shortest Job First maximize throughput by prioritizing quick tasks, but fairness may suffer.
- **Notation:** Denoted as **TP**.

Advanced Notes on Scheduling Times

- **Assumption (No I/O Operations):** Since processes have no I/O bursts, each process has a single CPU burst ($BT = CPU\ BT$). This simplifies calculations, as there's no time spent in Waiting/Blocked state for I/O.
- **Interdependencies:**
 - $TAT = WT + BT$ (since no I/O).
 - $RT \leq WT$ (response time is the first wait, while WT includes all waits in preemptive systems).
- **Metrics Trade-Offs:**
 - Minimizing WT may increase RT if long jobs are delayed.
 - Maximizing TP may increase TAT for longer processes.
- **Real-World Context:** In real systems with I/O, burst times alternate between CPU and I/O, complicating scheduling. The no-I/O assumption is common in academic exercises to focus on CPU scheduling algorithms.

CPU Scheduling: Types

CPU scheduling can be classified into two main types based on whether a running process can be interrupted: **Preemptive** and **Non-Preemptive**. These determine how the short-term scheduler allocates CPU time.

Preemptive Scheduling

- **Definition:** The OS can interrupt a running process to give the CPU to another process, even if the current process isn't finished.
- **For Beginners:** Imagine a chef cooking multiple dishes. If a priority order comes in (e.g., for a VIP), they pause one dish to start the new one. The paused dish waits until the chef returns.
- **How It Works:**
 - The scheduler can preempt (interrupt) a process due to:
 - **Time Slice Expiry:** In time-sharing systems (e.g., Round-Robin), each process gets a fixed time quantum (e.g., 10 ms). If time's up, it's sent back to the Ready Queue.
 - **Higher-Priority Process:** A new or higher-priority process arrives.
 - **Event Completion:** E.g., an I/O-bound process finishes I/O and becomes ready (not applicable here due to no I/O assumption).
 - The interrupted process returns to the **Ready Queue**, and its state (e.g., CPU registers) is saved in its PCB.
- **Example:** While a game runs, a system update arrives with higher priority. The scheduler pauses the game, runs the update, and later resumes the game.
- **Algorithms:**
 - **Round-Robin (RR):** Each process gets a time slice; fair but may increase context switch overhead.
 - **Priority Scheduling (Preemptive):** Higher-priority processes preempt lower ones; aging prevents starvation.
 - **Shortest Remaining Time First (SRTF):** Preempts if a new process has a shorter remaining burst time.
- **Advantages:**
 - Improves responsiveness (lowers response time for new processes).
 - Ensures fairness by sharing CPU time.
 - Ideal for time-sharing systems (e.g., modern OS like Windows, Linux).
- **Disadvantages:**
 - Context switch overhead (saving/restoring PCB) can reduce efficiency.
 - Complex to implement due to frequent interruptions.
- **Advanced:** Preemption requires precise timer interrupts (handled by hardware) and efficient PCB management. In multi-core systems, preemption may occur per core.

Non-Preemptive Scheduling

- **Definition:** Once a process gets the CPU, it runs to completion (or until it voluntarily yields, e.g., terminates). The OS cannot interrupt it.
- **For Beginners:** The chef finishes cooking one dish completely before starting the next, even if a VIP order arrives.
- **How It Works:**
 - The scheduler selects a process from the Ready Queue and lets it run until it finishes its CPU burst (or terminates, since no I/O).
 - No interruptions unless the process explicitly yields (e.g., completes).
- **Example:** A video rendering program runs fully before the CPU switches to a browser.
- **Algorithms:**
 - **First-Come-First-Served (FCFS):** Processes run in arrival order; simple but can cause long waits (convoy effect).
 - **Shortest Job First (SJF):** Picks the process with the shortest burst time; minimizes waiting time but risks starvation.
 - **Priority Scheduling (Non-Preemptive):** Runs the highest-priority process to completion.
- **Advantages:**
 - Simpler implementation (no context switch during execution).
 - Lower overhead since no preemption.
 - Suitable for batch systems where throughput matters more than responsiveness.
- **Disadvantages:**
 - Poor responsiveness (long response times if a long process runs first).
 - Risk of starvation for short/low-priority processes (e.g., convoy effect in FCFS).
- **Advanced:** Non-preemptive scheduling is less common in modern interactive OS but used in specific cases (e.g., embedded systems or batch processing). The no-I/O assumption aligns well with non-preemptive algorithms, as processes don't block voluntarily.

// Note:- Every Process Has No I/O Operations [Assumption In All Algorithms] Hence We Will Only Look At CPU Burst Time

Advanced Notes on Scheduling Types

- **No I/O Assumption Impact:**

- Without I/O operations, processes don't enter the Waiting/Blocked state, simplifying scheduling to focus on CPU bursts.
- Preemptive algorithms like Round-Robin still apply (e.g., time slicing), but non-preemptive algorithms like SJF are more effective since processes run to completion without interruptions.
- **Context Switching:**
 - Preemptive: Frequent context switches increase overhead but improve fairness/response time.
 - Non-Preemptive: Minimal context switches (only at completion), but long processes can delay others.
- **Real-World Use:**
 - **Preemptive:** Dominant in modern OS (e.g., Linux CFS, Windows). Essential for multitasking and interactive systems.
 - **Non-Preemptive:** Used in batch systems or real-time systems with predictable tasks.
- **Hybrid Approaches:** Modern OS combine both (e.g., Linux allows preemption but supports non-preemptive kernel tasks in specific modes).

Question Recap

The earlier question ("Which scheduler reduces the degree of multiprogramming?") was answered as the **Mid-Term Scheduler**, which swaps processes out to disk, reducing the number in memory.

This covers scheduling times and CPU scheduling types comprehensively, from basics (definitions) to advanced (algorithm impacts, trade-offs). Let me know if you need further details, examples, or specific algorithms like FCFS or Round-Robin explained!